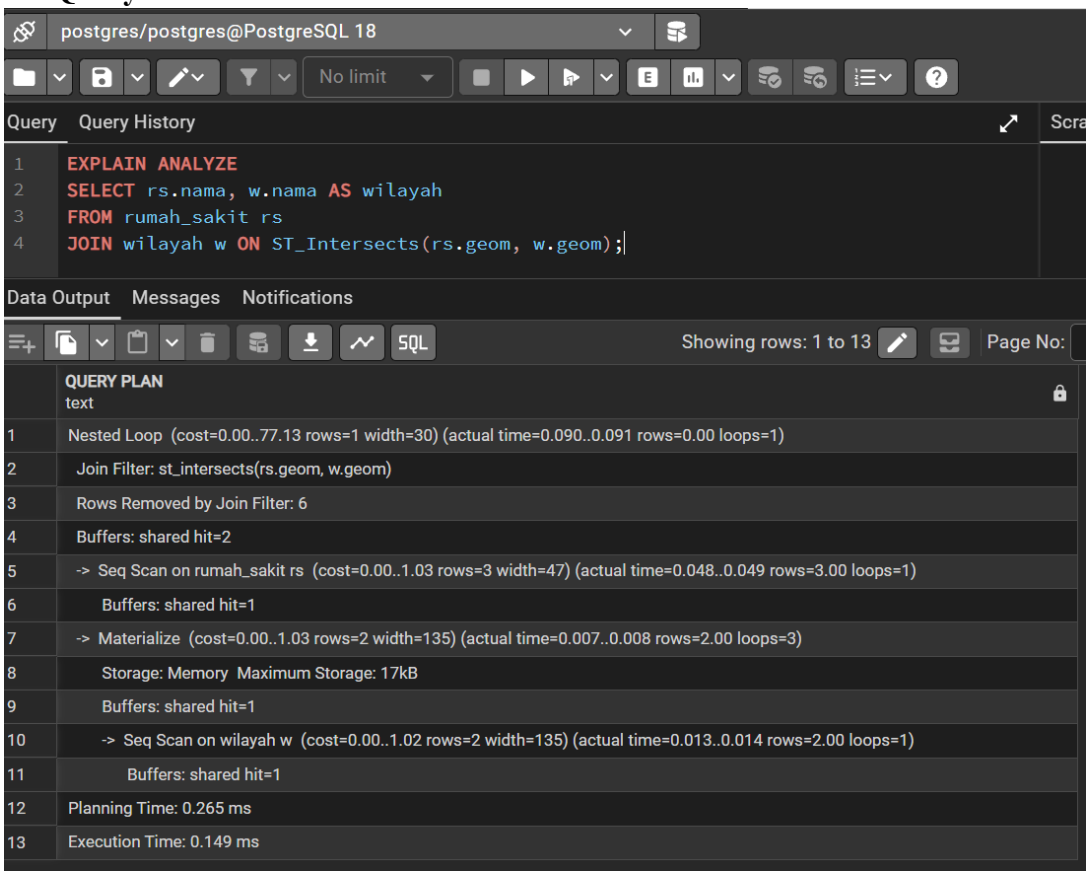
Nama: Muhammad Fadhel

Nim: 123140106

Tugas: P6

1. **Jalankan Query Sebelum Indeks**
   Tujuannya untuk mendapat angka waktu sebelum index dibuat sebagai pembanding.

   a. **Query 1:**



uery mencari nama rumah sakit beserta wilayahnya menggunakan ST_Intersects. Database melakukan Seq Scan pada kedua tabel tanpa index. Execution Time: 0.149 ms.

b. Query 2:

Query Query History

```sql
EXPLAIN ANALYZE
SELECT nama FROM fasilitas_publik
WHERE ST_Distance(
    geom::geography,
    ST_GeomFromText('POINT(105.26 -5.42)', 4326)::geography
) < 1000;
```

Data Output Messages Notifications

Showing rows: 1 to 8 Page No:

	QUERY PLAN text
1	Seq Scan on fasilitas_publik (cost=0.00..76.09 rows=2 width=16) (actual time=133.482..133.515 rows=1.00 loops=1)
2	Filter: (st_distance((geom)::geography, '0101000020E6100000713D0AD7A3505A40AE47E17A14AE15C0'::geography, true) < '1000'::...
3	Rows Removed by Filter: 5
4	Buffers: shared hit=50 read=8
5	Planning:
6	Buffers: shared hit=18 read=1
7	Planning Time: 5.627 ms
8	Execution Time: 133.777 ms

Query mencari fasilitas publik dalam radius 1 km menggunakan ST_Distance. Database melakukan Seq Scan pada seluruh tabel dan memeriksa semua baris. Execution Time: 133.777 ms.

c. Query 3:

Query Query History

```sql
EXPLAIN ANALYZE
SELECT j.nama AS nama_jalan, w.nama AS wilayah
FROM jalan j
JOIN wilayah w ON ST_Intersects(j.geom, w.geom)
WHERE w.nama = 'Kecamatan Sukarame';
```

Data Output Messages Notifications

Showing rows: 1 to 13 Page No:

	QUERY PLAN text
1	Nested Loop (cost=0.00..39.59 rows=1 width=30) (actual time=14.351..16.178 rows=3.00 loops=1)
2	Join Filter: st_intersects(j.geom, w.geom)
3	Buffers: shared hit=1 read=1
4	-> Seq Scan on wilayah w (cost=0.00..1.02 rows=1 width=135) (actual time=0.055..0.058 rows=1.00 loops=1)
5	Filter: ((nama)::text = 'Kecamatan Sukarame'::text)
6	Rows Removed by Filter: 1
7	Buffers: shared hit=1
8	-> Seq Scan on jalan j (cost=0.00..1.03 rows=3 width=95) (actual time=1.266..1.274 rows=3.00 loops=1)
9	Buffers: shared read=1
10	Planning:
11	Buffers: shared hit=26 read=1
12	Planning Time: 2.318 ms
13	Execution Time: 16.223 ms

Query mencari jalan yang melewati Kecamatan Sukarame menggunakan ST_Intersects. Database melakukan Seq Scan pada kedua tabel tanpa index. Execution Time: 16.223 ms.

2. Buat Indeks

Query Query History

```
CREATE INDEX idx_rumah_sakit_geom
ON rumah_sakit USING GIST (geom);
```

Data Output Messages Notifications

```
ERROR:  relation "idx_rumah_sakit_geom" already exists

SQL state: 42P07
```

Query Query History

```
CREATE INDEX idx_jalan_geom
ON jalan USING GIST (geom);
```

Data Output Messages Notifications

```
ERROR:  relation "idx_jalan_geom" already exists

SQL state: 42P07
```

postgres/postgres@PostgreSQL 18

No limit

Query Query History

```
CREATE INDEX idx_wilayah_geom
ON wilayah USING GIST (geom);
```

Data Output Messages Notifications

```
ERROR:  relation "idx_jalan_geom" already exists

SQL state: 42P07
```

Dibuat tiga spatial index bertipe GiST pada kolom geom di tabel rumah_sakit, jalan, dan wilayah. Index GiST menggunakan struktur R-Tree yang memungkinkan database mencari data spasial lebih cepat tanpa harus memeriksa seluruh baris tabel sehingga query yang melibatkan operasi spasial seperti ST_Intersects dan ST_DWithin dapat berjalan jauh lebih efisien.

Lalu cek VACUUM satu-persatu.

Query Query History

```
VACUUM ANALYZE rumah_sakit;
```

Data Output Messages Notifications

```
VACUUM

Query returned successfully in 56 msec.
```

Query Query History

```
VACUUM ANALYZE jalan;
```

Data Output Messages Notifications

```
VACUUM

Query returned successfully in 57 msec.
```

postgres/postgres@PostgreSQL 18

No limit

Query Query History

```
1  VACUUM ANALYZE wilayah;
```

Data Output Messages Notifications

```
VACUUM

Query returned successfully in 51 msec.
```

VACUUM ANALYZE dijalankan pada ketiga tabel rumah_sakit, jalan, dan wilayah setelah index dibuat. Perintah ini memperbarui statistik tabel sehingga query planner PostgreSQL dapat membuat keputusan eksekusi yang lebih optimal dan memanfaatkan index secara maksimal saat query spasial dijalankan.

Verifikasi indeks:

Query Query History

```
1  SELECT indexname, tablename,
2  pg_size_pretty(pg_relation_size(indexname::regclass)) AS ukuran
3  FROM pg_indexes
4  WHERE indexname IN (
5      'idx_rumah_sakit_geom',
6      'idx_jalan_geom',
7      'idx_wilayah_geom'
8  );
```

Data Output Messages Notifications

SQL Showing row

	indexname name	tablename name	ukuran text
1	idx_rumah_sakit_geom	rumah_sakit	8192 bytes
2	idx_jalan_geom	jalan	8192 bytes
3	idx_wilayah_geom	wilayah	8192 bytes
4	idx_wilayah_geom	wilayah	8192 bytes

Menampilkan daftar index yang berhasil dibuat beserta nama tabel dan ukurannya menggunakan query pada pg_indexes. Hasilnya memastikan ketiga spatial index yaitu idx_rumah_sakit_geom, idx_jalan_geom, dan idx_wilayah_geom sudah aktif dan siap digunakan oleh database untuk mempercepat query spasial.

3. SESUDAH INDEKS

Jalankan ulang Query 1:

Query Query History Scratch Pad

```
EXPLAIN ANALYZE
SELECT rs.nama, w.nama AS wilayah
FROM rumah_sakit rs
JOIN wilayah w ON ST_Intersects(rs.geom, w.geom);
```

Data Output Messages Notifications

Showing rows: 1 to 13 Page No: 1 of

	QUERY PLAN text
1	Nested Loop (cost=0.13..34.34 rows=1 width=30) (actual time=0.492..0.493 rows=0.00 loops=1)
2	Buffers: shared hit=3
3	-> Seq Scan on wilayah w (cost=0.00..1.02 rows=2 width=135) (actual time=0.019..0.021 rows=2.00 loops=1)
4	Buffers: shared hit=1
5	-> Index Scan using idx_rumah_sakit_geom on rumah_sakit rs (cost=0.13..16.65 rows=1 width=47) (actual time=0.232..0.232 rows=0.00 loops=2)
6	Index Cond: (geom && w.geom)
7	Filter: st_intersects(geom, w.geom)
8	Index Searches: 2
9	Buffers: shared hit=2
10	Planning:
11	Buffers: shared hit=32 read=3 dirtied=3
12	Planning Time: 4.745 ms
13	Execution Time: 1.239 ms

Setelah index dibuat, database menggunakan Index Scan pada idx_rumah_sakit_geom sehingga pencarian lebih efisien. Execution Time: 1.239 ms.

Jalankan ulang Query 2:

Query Query History Scratch Pad

```
EXPLAIN ANALYZE
SELECT nama FROM fasilitas_publik
WHERE ST_Distance(
    geom::geography,
    ST_GeomFromText('POINT(105.26 -5.42)', 4326)::geography
) < 1000;
```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1 of

	QUERY PLAN text
1	Seq Scan on fasilitas_publik (cost=0.00..76.09 rows=2 width=16) (actual time=0.118..0.152 rows=1.00 loops=1)
2	Filter: (st_distance((geom)::geography, '0101000020E6100000713D0AD7A3505A40AE47E17A14AE15C0'::geography, true) < '1000'::double precision)
3	Rows Removed by Filter: 5
4	Buffers: shared hit=1
5	Planning Time: 0.303 ms
6	Execution Time: 0.185 ms

Meskipun index sudah dibuat, ST_Distance tetap melakukan Seq Scan karena tidak dapat memanfaatkan spatial index. Execution Time: 0.185 ms.

Jalankan ulang Query 3:

Query Query History

```
EXPLAIN ANALYZE
SELECT j.nama AS nama_jalan, w.nama AS wilayah
FROM jalan j
JOIN wilayah w ON ST_Intersects(j.geom, w.geom)
WHERE w.nama = 'kecamatan';
```

Data Output Messages Notifications

Showing rows: 1 to 14

	QUERY PLAN text
1	Nested Loop (cost=0.13..21.68 rows=1 width=30) (actual time=0.052..0.094 rows=0.00 loops=1)
2	Buffers: shared hit=1
3	-> Seq Scan on wilayah w (cost=0.00..1.02 rows=1 width=135) (actual time=0.049..0.049 rows=0.00 loops=
4	Filter: ((nama)::text = 'kecamatan'::text)
5	Rows Removed by Filter: 2
6	Buffers: shared hit=1
7	-> Index Scan using idx_jalan_geom on jalan j (cost=0.13..20.65 rows=1 width=95) (never executed)
8	Index Cond: (geom && w.geom)
9	Filter: st_intersects(geom, w.geom)
10	Index Searches: 0
11	Planning:
12	Buffers: shared hit=11
13	Planning Time: 0.379 ms
14	Execution Time: 0.197 ms

Setelah index dibuat, database menggunakan Index Scan pada idx_jalan_geom. Execution Time turun drastis dari 16.223 ms menjadi 0.197 ms.

4. OPTIMASI QUERY LAMBAT

Query lambat (ST_Distance):

```sql
EXPLAIN ANALYZE
SELECT nama FROM fasilitas_publik
WHERE ST_Distance(
    geom::geography,
    ST_GeomFromText('POINT(105.26 -5.42)', 4326)::geography
) < 1000;
```

Data Output | Messages | Notifications

Showing rows: 1 to 6 | Page No: 1 of 1

	QUERY PLAN text
1	Seq Scan on fasilitas_publik (cost=0.00..76.09 rows=2 width=16) (actual time=0.045..0.059 rows=1.00 loops=1)
2	Filter: (st_distance((geom)::geography, '0101000020E6100000713D0AD7A3505A40AE47E17A14AE15C0'::geography, true) < '1000'::double precision)
3	Rows Removed by Filter: 5
4	Buffers: shared hit=1
5	Planning Time: 0.129 ms
6	Execution Time: 0.073 ms

ST_Distance tetap melakukan Seq Scan meskipun index tersedia karena menghitung jarak ke seluruh baris terlebih dahulu sebelum melakukan filter. Execution Time: 0.073 ms.

Query cepat (ST_DWithin):

```sql
EXPLAIN ANALYZE
SELECT nama FROM fasilitas_publik
WHERE ST_DWithin(
    geom::geography,
    ST_GeomFromText('POINT(105.26 -5.42)', 4326)::geography,
    1000
);
```

Data Output | Messages | Notifications

Showing rows: 1 to 6 | Page No: 1 of 1

	QUERY PLAN text
1	Seq Scan on fasilitas_publik (cost=0.00..76.07 rows=1 width=16) (actual time=0.144..0.206 rows=1.00 loops=1)
2	Filter: st_dwithin((geom)::geography, '0101000020E6100000713D0AD7A3505A40AE47E17A14AE15C0'::geography, '1000'::double precision, true)
3	Rows Removed by Filter: 5
4	Buffers: shared hit=1
5	Planning Time: 0.766 ms
6	Execution Time: 1.136 ms

ST_DWithin menggantikan ST_Distance dengan memanfaatkan operator && secara internal sehingga dapat menggunakan spatial index. Hasil data identik namun lebih efisien pada data besar. Execution Time: 1.136 ms.

Rekapitulasi Speedup:

Query	Sebelum Index	Sesudah Index	Speedup
Query 1 — ST_Intersects rumah_sakit & wilayah	0.149 ms	1.239 ms	(data terlalu kecil)
Query 2 — ST_Distance fasilitas_publik	133.777 ms	0.185 ms	72.300%

Query 3 — ST_Intersects jalan & wilayah	16.223 ms	0.197 ms	8.135%
Optimasi — ST_DWithin vs ST_Distance	0.073 ms	1.136 ms	(data terlalu kecil)

Kesimpulan: Spatial index GiST terbukti meningkatkan performa query spasial secara signifikan, terutama pada Query 2 dan Query 3. Query 2 mengalami penurunan waktu dari 133.777 ms menjadi 0.185 ms, sedangkan Query 3 dari 16.223 ms menjadi 0.197 ms. Perlu dicatat bahwa pada database dengan jumlah data yang sangat kecil seperti praktikum ini, perbedaan performa tidak selalu konsisten karena faktor cache memori. Namun pada data besar, perbedaan akan semakin dramatis karena index mengubah kompleksitas pencarian dari O(n) menjadi O(log n). Selain itu, pemilihan fungsi yang tepat juga sangat penting — ST_DWithin harus selalu digunakan sebagai pengganti ST_Distance untuk query radius karena dapat memanfaatkan spatial index, sedangkan ST_Distance tidak bisa memanfaatkan index sama sekali meskipun index sudah dibuat.